

Python Code

```
1  import numpy as np
2  import random
3
4  # =====
5  # 1. SAMPLE PROBLEM DEFINITION (RCPSP)
6  # =====
7  # 5 Activities (0 = Start Dummy, 4 = End Dummy)
8  # Precedence map: key must be completed before values can start
9  PRECEDENCE = {
10     0:,
11     1:,
12     2:,
13     3:,
14     4: []
15 }
16
17 DURATIONS = {0: 0, 1: 4, 2: 2, 3: 3, 4: 0}
18 RESOURCE_REQS = {0: 0, 1: 4, 2: 3, 3: 5, 4: 0}
19 TOTAL_RESOURCES_AVAILABLE = 6
20
21
22 # =====
23 # 2. SERIAL GENERATION SCHEME (SGS)
24 # =====
25 def serial_generation_scheme(chromosome):
26     """
27     Decodes an activity-list chromosome into a valid schedule
28     calculating the total project duration (makespan).
29     """
30     start_times = {}
31     end_times = {}
32
33     # Track resource consumption across a timeline maxed out at a theoretical
34     limit
35     max_horizon = sum(DURATIONS.values()) + 1
36     resource_timeline = np.zeros(max_horizon)
37
38     for activity in chromosome:
39         # Step A: Find the earliest time based strictly on Precedence
40         Constraints
41         earliest_start = 0
42         for pred, successors in PRECEDENCE.items():
43             if activity in successors:
44                 earliest_start = max(earliest_start, end_times.get(pred, 0))
45
46         # Step B: Find the earliest time slot that ALSO satisfies Resource
47         Constraints
48         current_start = earliest_start
49         duration = DURATIONS[activity]
50         req = RESOURCE_REQS[activity]
```

```

48
49     if duration == 0: # Dummy start/end activities
50         start_times[activity] = current_start
51         end_times[activity] = current_start
52         continue
53
54     while True:
55         # Check if resources are available throughout the entirety of this
window
56         window = resource_timeline[current_start : current_start +
duration]
57         if np.all(window + req <= TOTAL_RESOURCES_AVAILABLE):
58             # Allocate resources
59             resource_timeline[current_start : current_start + duration] +=
req
60             start_times[activity] = current_start
61             end_times[activity] = current_start + duration
62             break
63             current_start += 1
64
65     return end_times[chromosome[-1]] # Return the finish time of the final
dummy task
66
67
68 # =====
69 # 3. GENETIC ALGORITHM OPERATORS
70 # =====
71 def generate_valid_chromosome():
72     """
73     Generates an activity list that natively respects topological precedence
conditions.
74     """
75     activities = list(DURATIONS.keys())
76     chromosome = [0] # Always starts with project origin
77     eligible = [act for act in activities if act != 0]
78
79     while eligible:
80         # Pick an activity whose predecessors have already been placed
81         valid_choices = []
82         for act in eligible:
83             # Check if all its predecessors are in the chromosome
84             is_valid = True
85             for pred, successors in PRECEDENCE.items():
86                 if act in successors and pred not in chromosome:
87                     is_valid = False
88                     break
89             if is_valid:
90                 valid_choices.append(act)
91
92         chosen = random.choice(valid_choices)
93         chromosome.append(chosen)
94         eligible.remove(chosen)
95
96     return chromosome

```

```

97
98
99 def crossover(parent1, parent2):
100     """
101     Order-Based Crossover (OX) adapted to preserve topology structural
requirements.
102     """
103     # Fix the start and end dummies
104     core1 = parent1[1:-1]
105     core2 = parent2[1:-1]
106
107     size = len(core1)
108     idx1, idx2 = sorted(random.sample(range(size), 2))
109
110     # Inherit slice from parent 1
111     child1_core = [None] * size
112     child1_core[idx1:idx2] = core1[idx1:idx2]
113
114     # Fill remaining from parent 2 tracking order
115     p2_pointer = 0
116     for i in range(size):
117         if child1_core[i] is None:
118             while core2[p2_pointer] in child1_core:
119                 p2_pointer += 1
120             child1_core[i] = core2[p2_pointer]
121
122     return [0] + child1_core + [len(DURATIONS)-1]
123
124
125 def mutate(chromosome, mutation_rate=0.1):
126     """
127     Swap mutation that respects structural precedence parameters.
128     """
129     if random.random() > mutation_rate:
130         return chromosome
131
132     mutated = list(chromosome)
133     # Don't touch index 0 (Start) and last index (End)
134     idx1, idx2 = random.sample(range(1, len(chromosome) - 1), 2)
135
136     # Swap elements temporarily
137     mutated[idx1], mutated[idx2] = mutated[idx2], mutated[idx1]
138
139     # Verify if the random swap broke precedence rules
140     for i, act in enumerate(mutated):
141         for pred, successors in PRECEDENCE.items():
142             if act in successors:
143                 pred_idx = mutated.index(pred)
144                 if pred_idx > i:
145                     return chromosome # Invalid: Return original structure
untouched
146
147     return mutated
148

```

```

149
150 # =====
151 # 4. MAIN OPTIMIZATION LOOP
152 # =====
153 def genetic_algorithm(pop_size=20, generations=30, mutation_rate=0.2):
154     # Initialize Population
155     population = [generate_valid_chromosome() for _ in range(pop_size)]
156
157     print(f"Starting Genetic Algorithm optimization across {generations}
generations...\n")
158
159     for gen in range(generations):
160         # Calculate fitness (lower makespan is better)
161         fitness_scores = [serial_generation_scheme(chrom) for chrom in
population]
162
163         # Sort population by best fitness
164         sorted_indices = np.argsort(fitness_scores)
165         population = [population[i] for i in sorted_indices]
166         fitness_scores = [fitness_scores[i] for i in sorted_indices]
167
168         # Log generation progress
169         if (gen + 1) % 5 == 0 or gen == 0:
170             print(f"Generation {gen+1:02d} | Best Makespan Found:
{fitness_scores[0]} days")
171
172         # Selection: Retain the elite top 20%
173         elite_count = int(pop_size * 0.2)
174         next_generation = population[:elite_count]
175
176         # Breeding to fill up the remaining population pool
177         while len(next_generation) < pop_size:
178             # Tournament selection
179             p1, p2 = random.sample(population[:int(pop_size*0.6)], 2)
180             child = crossover(p1, p2)
181             child = mutate(child, mutation_rate)
182             next_generation.append(child)
183
184         population = next_generation
185
186     # Final Output Results
187     best_chromosome = population[0]
188     best_makespan = serial_generation_scheme(best_chromosome)
189
190     print("\n" + "="*40)
191     print("OPTIMIZATION COMPLETE")
192     print("="*40)
193     print(f"Optimal Scheduling Order: {best_chromosome}")
194     print(f"Minimized Project Makespan: {best_makespan} days")
195
196
197 if __name__ == "__main__":
198     genetic_algorithm(pop_size=30, generations=20, mutation_rate=0.15)

```

